



SQL – Sub Query

SUBQUERIES

- ▶ SQL provides a mechanism for the nesting of subqueries. A **subquery** is a select-from-where expression that is nested within another query.

- ▶ The nesting can be done in the following SQL query
select $A_1, A_2, \dots, A_n$ from $r_1, r_2, \dots, r_m$ where P

as follows:

- ▶ From clause: r_i can be replaced by any valid subquery
- ▶ Where clause: P can be replaced with an expression of the form:

B <operation> (subquery)

B is an attribute and <operation> to be defined later.

- ▶ Select clause:

A_i can be replaced by a subquery that generates a single value.

SUBQUERIES

- ▶ Most of the time tasks can be achieved directly with existing SQL commands, but having subqueries can save storage space, reduce the query length and/or complexity, and also provides another way to give commands
- ▶ Common use of subqueries:
 - ▶ Perform tests for set membership
 - ▶ Make set comparisons
 - ▶ Determine set cardinalities

SET MEMBERSHIP

- ▶ SQL allows testing tuples for membership in a set/relation
 - ▶ That is, to check if a specific value exists in the table or not
- ▶ The connectives used to check for the membership:
 - ▶ IN
 - ▶ Not IN

SET MEMBERSHIP

▶ *IN*

- ▶ This connective returns True if any desired value exists in the relation
- ▶ Used with subqueries that are placed in the *where* clause
- ▶ General form:
 - ▶ A_1 in r_1
 - ▶ Here r_1 can be any pre-defined relation or a relation being deduced from a subquery

SET MEMBERSHIP

▶ *Not IN*

- ▶ This connective returns True if specified value is not present in the relation
- ▶ Mostly used with subqueries that are placed in the *where* clause
- ▶ General form:
 - ▶ A_1 not in r_1
 - ▶ Here r_1 can be any pre-defined relation or a relation being deduced from a subquery

SET MEMBERSHIP

- ▶ For either of the connectives, enumerated set of attributes can be compared at the same time as well
 - ▶ For example, $(A_1, A_2, \dots, A_n)$ in r_1

SET MEMBERSHIP

	course_id
1	CS-101

- Find courses offered in Fall 2017 and in Spring 2018
- Can use set intersection like before but using subqueries here as:

```

select distinct course_id
from section
where semester = 'Fall' and year=
2017 and
      course_id in (select course_id
                     from section
                     where semester =
'Spring' and year= 2018);

```

<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>room_number</i>	<i>time_slot_id</i>
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Figure 2.6 The *section* relation.

SET MEMBERSHIP

	course_id
1	CS-347
2	PHY-101

- Find courses offered in Fall 2017 but not in Spring 2018

```

select distinct course_id
from section
where semester = 'Fall' and
year= 2017 and
       course_id not in
(select course_id
      from
section
      where
semester = 'Spring' and
year= 2018);
    
```

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Figure 2.6 The *section* relation.

SET MEMBERSHIP

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	null

Figure 4.2 The *takes* relation.

- Find the total number of (distinct) students who have taken course sections taught by the instructor with *ID* 10101
- You may solve this by using join or cartesian product, but using subqueries here:

```

select count (distinct ID)
from takes
where (course_id, sec_id, semester, year) in
           (select course_id, sec_id, semester,
year
from teaches
where teaches.ID= 10101);
    
```

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Figure 2.7 The *teaches* relation.

count (distinct ID)
6

SET COMPARISON

- ▶ SQL allows comparing values in two sets using subqueries
 - ▶ That is, to check if they are equal, less than, greater than etc from each other
- ▶ The operations used to do the comparison:
 - ▶ *Some*
 - ▶ *all*

SET COMPARISON

▶ *some*

- ▶ This operation refers to any comparison with at least one value
- ▶ Used with subqueries that are placed in the *where* clause
- ▶ General form:
 - ▶ {Comparison operator} **some**
 - ▶ For example, > **some**, <= **some**, <> **some**, etc
 - ▶ This means that the value is greater than at least one, or less than or equal to at least one, or not equal to at least one, respectively

SET COMPARISON

▶ *all*

- ▶ This operation refers to any comparison for all values in a set
- ▶ Mostly used with subqueries that are placed in the *where* clause
- ▶ General form:
 - ▶ {Comparison operator} all
 - ▶ For example, > all, <= all, <> all, etc
 - ▶ This means that the value is greater than all the values in the relation, or less than or equal to all the values, or equals to nothing, respectively

SET COMPARISON

- Find names of instructors with salary greater than that of some (**at least one**) instructor in the Computer Science department, using subqueries

```
select name  
from instructor  
where salary > some (select salary  
                        from instructor  
                        where dept_name =  
                        'Comp. Sci.');
```

	name
1	Brandt
2	Einstein
3	Gold
4	Katz
5	Kim
6	Singh
7	Wu
8	Crick

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

SET COMPARISON

- Find the names of all instructors whose salary is greater than the salary of each instructor in the Computer Science department.

select *name*

from *instructor*

where *salary* > **all** (**select** *salary*
from *instructor*
where *dept name* =
 'Comp. Sci.');

	<i>name</i>
1	Einstein

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

EXISTS CLAUSE

- ▶ SQL allows using ***exists*** and ***not exists*** in place of ***in/not in*** connectives
- ▶ They return True or False depending on the condition
 - ▶ ***exists***
 - ▶ This returns True if the argument subquery produces a non-empty relation
 - ▶ That is, the resultant relation has at least one tuple in it
 - ▶ ***not exists***
 - ▶ This returns True if the argument subquery produces an empty relation
 - ▶ That is, the resultant relation has no tuples in it

EXISTS CLAUSE

- ▶ Subqueries allow user to refer to the relation name from an outer query inside the inner query, as these tables might be related to each other for the querying purposes
- ▶ Such relation is called a *correlation*, that is, the outer query table is a correlation to the inner query table
- ▶ Any subquery that uses the reference to its correlation is called a *correlated subquery*
- ▶ For example:

```
select course_id  
from section as S  
where course_id in (select * from course as T  
                    where S.course_id = T.course_id);
```
- ▶ Here, S is a correlation to T, and the inner query is correlated subquery

EXISTS CLAUSE

- ▶ Another way of specifying the query “Find all courses taught in both the Fall 2017 semester and in the Spring 2018 semester”

```
select course_id
from section as S
where semester = 'Fall' and year = 2017 and
      exists (select *
              from section as T
              where semester = 'Spring' and year = 2018
                and S.course_id = T.course_id);
```

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Figure 2.6 The *section* relation.

- ▶ **Correlation name?**
 - ▶ variable S in the outer query
- ▶ **Correlated subquery?**
 - ▶ the inner query

	course_id
1	CS-101

EXISTS CLAUSE

► Not Exists Example

```
select course_id
  from section as S
 where semester = 'Fall' and year = 2017 and
        not exists (select *
                    from section as T
                   where semester = 'Spring' and year = 2018
                      and S.course_id = T.course_id);
```

SUBQUERIES IN THE FROM CLAUSE

- Find the average instructors' salaries of those departments where the average salary is greater than \$42,000."

```
select dept_name, avg_salary
from ( select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name)
where avg_salary > 42000;
```

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333.3333333333
Elec. Eng.	80000
Finance	85000
History	61000
Physics	91000

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

THE WITH CLAUSE

- ▶ The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the **with** clause occurs.

WITH CustomerTotals AS (

 SELECT Customer, SUM(Amount) AS TotalAmount

 FROM Orders

 GROUP BY Customer

)

SELECT *

FROM CustomerTotals

WHERE TotalAmount > 400;

SCALAR SUBQUERY

- ▶ Scalar subquery is one which is used where a single value is expected.
- ▶ List all departments along with the number of instructors in each department

```
select dept_name,  
       ( select count(*)  
         from instructor  
         where department.dept_name = instructor.dept_name)  
       as num_instructors  
from department;
```

SCALAR SUBQUERY

- ▶ Can be achieved through Joins or other ways, but serves efficient and simple in terms of execution. Saves runtime storage and space.
- ▶ Sub query will show runtime error if result has more than 1 value.

THE CASE STATEMENT

- ▶ The general form of the **case** statement is as follows:

```
case
  when pred1 then result1
  when pred2 then result2
  .....
  when predn then resultn
  else result0
end
```

- ▶ The operation returns $result_i$, where i is the first of $pred1, pred2, \dots, predn$ that is satisfied; if none of the predicates is satisfied, the operation returns $result0$.
- ▶ Case statements can be used in any place where a value is expected

THE CASE STATEMENT

- For each instructor ID and name, assign an attribute called `well_paid`, that will have 'Yes' value if their salary is above \$60,000 and 'No' otherwise

```
select ID, name,
(case
    when salary > 60000 then 'Yes'
    else 'No'
end) as well_paid
from instructor;
```

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.



THANK YOU